



OOPS LAB PRESENTATION

GROUP MEMBERS:

- **SANIA ARSHAD (2023-BCS-098)**
- **EMAN WAHEED (2023-BCS-027)**
- **AMNA ABID (2023-BCS-007)**
- **Arshiya Shakeel(2023-BCS-017)**
- **Zahra Bashir(2023-BCS-092)**
- **Moomna Atiq(2023-BCS-065)**



TOPIC: SOCIAL MEDIA PLATFORM MANAGEMENT SYSTEM

OVERVIEW:

In the digital age, social media has become the cornerstone of human interaction, connecting individuals across the globe. The Social Media Platform Management System is an intricate tapestry of technology, designed to foster these connections, allowing users to share their stories, express their thoughts, and build vibrant online communities. This system encapsulates the essence of modern communication, blending functionality with elegance to create a seamless user experience.

A social media platform management system in a database typically involves organizing and managing various entities and their relationships to support functionalities such as user management, content management, interactions, and analytics.



Designing :

- When designing a social media platform management system using Object-Oriented Programming (OOP), the goal is to model the real-world entities (users, posts, comments, etc.) as objects with attributes and behaviors. Below is an overview of how such a system might be structured in an OOP project, including key classes and their relationships.

- **Key Classes:**

- USERS
- FRIENDS
- POSTS
- LIKES
- MESSAGES
- PROFILE
- COMMENTS

Attributes:

- **USERS:** User consist of userid(PK), name and password.
- **FRIENDS:** Friend consist of status, userid(FK) and friendid(PK).
- **POSTS:** Post consist of userid(FK), content and id(PK).
- **LIKES:** Likes consist of id(PK), userid(FK) and photoid(FK).
- **MESSAGES:** Message consist of id(PK),date sent and receiver id(FK).
- **PROFILE:** Profile consist of id(PK),userid(FK) and bio.
- **COMMENTS:** Comments consist of id(PK), postid(PK),date posted id and content.

Relationships Between Classes:

- **User and Post:** A user can create multiple posts. – Relationship: One-to-Many (User → Posts)
- **Post and Comment:** A post can have multiple comments. – Relationship: One-to-Many (Post → Comments)–
- **User and Comment:** A user can make multiple comments. – Relationship: One-to-Many (User → Comments)
- **User and Like:** A user can like multiple posts. – Relationship: One-to-Many (User → Likes)
- **Post and Like:** A post can receive multiple likes. – Relationship: One-to-Many (Post → Likes)
- **User and Share:** A user can share multiple posts. – Relationship: One-to-Many (User → Shares)
- **User and Message:** A user can send and receive multiple messages. – Relationship: One-to-Many (User → Sent Messages, User → Received Messages)

CONCEPTS IN JAVA:

ENCAPSULATION:

Encapsulation is the concept of bundling data (variables) and methods (functions) that operate on the data into a single unit or class. It restricts direct access to some of the object's components, which is a means of preventing unintended interference and misuse of the methods and data. Encapsulation is achieved through access modifiers like private, protected, and public.

Constructor:

A special method that is called when an object is instantiated. It has the same name as the class and does not have a return type.

Access Modifiers:

Keywords that set the access level for classes, variables, methods, and constructors. The four access levels are private, default (no modifier), protected, and public.

- **Abstract Class:**

An abstract class in Java cannot be instantiated and is declared with the abstract keyword. It can have both abstract methods (methods without a body) and concrete methods (methods with a body). Subclasses are expected to provide implementations for the abstract methods.

- **Super Keyword:**

The super keyword in Java is used within a subclass to refer to its superclass. It is used to call the superclass's methods and constructors.

- **Inheritance:**

Inheritance is a mechanism wherein a new class inherits properties and behavior (methods) from an existing class. The class that is inherited from is called the parent or superclass, and the class that inherits is called the child or subclass.

- **Static Keyword:**

Used to indicate that a member belongs to the class itself rather than to any specific instance. Static methods and variables can be accessed without creating an instance of the class.

- **Final Keyword:**

Used to declare constants, prevent method overriding, and inheritance.

- **Exception Handling:**

Mechanism to handle runtime errors using try, catch, finally, throw, and throws.

- **Interfaces:**

An interface in Java is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types. Interfaces cannot have instance fields. A class that implements an interface must provide implementations for all of its methods.

Polymorphism:

Polymorphism allows objects to be treated as instances of their parent class rather than their actual class. The two types of polymorphism are compile-time (method overloading) and runtime (method overriding).

overloading and overriding:

In Java, overloading and overriding are two key concepts related to polymorphism that allow methods to be defined in different ways within a class hierarchy.

Method Overloading:

Method overloading occurs when multiple methods in the same class have the same name but different parameter lists (different type, number, or both).

Method Overriding:

Method overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass. The method in the subclass must have the same name, return type, and parameters as the method in the superclass.

A piece of white paper with a torn, deckled edge is positioned horizontally in the center of the frame. The paper is set against a dark teal, textured background. On the left side of the paper, there are two small, dark, rectangular marks that look like staple indentations. The text "ANY QUESTIONS?" is printed in a bold, red, serif font, centered on the white paper.

ANY QUESTIONS?

*Thank
You*

